Customer Personalization in Digital Banking Using Behavioural Embeddings

by

Van Ly Nguyen

A dissertation submitted in partial fulfillment of the requirements
for the degree of

Master of Science

in

Data Analytics

Dublin Business School

Supervior: Bolu Adeagbo

January 2026

DECLARATION

I, Van Ly Nguyen, confirm that this research was independently completed by me in fulfillment of the requirements for the Master of Science in Data Analytics. This work is entirely original and has not been submitted to any other institution or published elsewhere. Furthermore, I affirm that the research adheres to ethical academic standards and contains no information that could cause harm. This project fully complies with the academic integrity policy of Dublin Business School.

Signed: Van Ly Nguyen

Student Number: 20039850

Date: 07/01/2025

Table of Contents

Abstract 7
Chapter 1: Introduction 8
1.1 Background and Context 8
1.2 Motivation 8
1.3 Problem Statement 9
1.4 Research Aim and Objectives 9
1.5 Research Scope 11
1.6 Significance and Expected Contributions 11
1.7 Project Structure/Outline 13
Chapter 2: Literature Review 14
2.1 Introduction 14
2.2 Ramya and Srinivas (2019): Big Data Analytics for Customer Segmentation 14
2.3 Saraswat et al. (2023): Conceptual Perspectives on AI in Fintech 15
2.4 Taneja (2024): Behavioral Clustering in Digital Banking 16
2.5 Kaur and Saini (2020): Ethics and Personalization in Digital Banking 16
2.6 Sy (2025): Contextual Bandits in Adaptive Recommendation 17
2.7 Synthesis of Research Gaps and Direction of the Present Study 17
Chapter 3: Research Plan, Methodology and Design 19
3.1 Introduction 19
3.2 ASUM-DM Methodological Framework 19
3.3 Analyse Phase 20
3.4 Design Phase 21
3.5 Configure and Build Phase 22
3.6 Deployment Phase 22
3.7 Operate and Optimize Phase 23
Chapter 4: Application Development and Results 24
4.1 Overview 24
4.2 Data Preprocessing 24
4.3 Behavioral Embedding Module 28

4.4 XGBoost Ranking Module 31
4.5 Explainability Module with SHAP 35
4.6 Contextual Bandit Module (Future Extension) 37
Chapter 5: Conclusions and Future Work 40
5.1 Conclusion 40
5.2 Limitations 40
5.3 Future work 41
References: 42
Appendix 42

Figure 1: ASUM-DM Methodology Overview (Source: ResearchGate, 2020) 20

Figure 2: Dataset map 25
Figure 3: Merging core tables 26
Figure 4: Correlation heatmap 27
Figure 5: Extract Embeddings 31
Figure 6: XGBoost training result 32
Figure 7: Evaluation after SMOTE 33
Figure 8: Evaluation after Hyperparameter Tuning 33
Figure 9: Confusion Matrix 34
Figure 10: Global SHAP plots 35
Figure 11: Local SHAP plots 36
Figure 12: Contextual Bandit results 37

Abstract

This research presents an adaptive recommendation framework for personalized financial product offerings using customer transaction behavior data. Leveraging a combination of deep learning and machine learning techniques, the system integrates four core modules: behavioral embedding via autoencoders, ranking with XGBoost, model explainability through SHAP, and dynamic decision-making using contextual bandits. The Czech banking dataset was used to simulate real-world digital banking environments, where user behavior was encoded into dense representations to capture latent financial patterns. The XGBoost model demonstrated improved classification performance after applying SMOTE for class balancing and hyperparameter tuning, achieving a recall of 0.82 on the positive class. SHAP analysis provided global and local interpretability by revealing the most influential latent features behind model predictions. Finally, the contextual bandit component, implemented with LinUCB, enabled adaptive exploration-exploitation strategies, optimizing content delivery over simulated user interactions. This end-to-end pipeline demonstrates the potential of integrating behavioral embeddings with interpretable and adaptive AI models to support more effective and transparent personalized financial recommendations. Future work will focus on deploying the system in a live setting and further enhancing the interpretability of latent representations.

Chapter 1: Introduction

1.1 Background and Context

The financial services industry has rapidly embraced digital transformation, making digital banking the primary channel for customer transactions (McKinsey & Company, 2021). This shift has enabled banks to collect extensive behavioral data and raised customer expectations for personalized, seamless experiences similar to those offered by leading technology firms (Kaur & Saini, 2020).

However, most financial institutions still rely on static segmentation models based on demographics, which often fail to reflect the dynamic nature of individual customer behaviors and preferences (Ramya & Srinivas, 2019). As a result, customers frequently receive irrelevant or poorly timed recommendations.

While AI and machine learning have delivered significant benefits in areas like fraud detection, their potential for adaptive, customer-facing personalization remains largely untapped (Alam & Islam, 2019). Current systems tend to lack transparency, adaptability, and regulatory compliance, especially given increasing demands for explainability and fairness, such as those outlined in GDPR.

Recent machine learning advances-including behavioral embeddings, learning-to-rank algorithms like XGBoost, and contextual bandit methods-offer promising solutions for building more adaptive and ethical personalization systems (Chen & Guestrin, 2016; Li et al., 2010)

1.2 Motivation

The limitations of current personalization methods within banking analytics have become increasingly apparent. Industry reports and empirical analyses consistently indicate inefficiencies such as low conversion rates in marketing campaigns, high rates of product application abandonment, and customer dissatisfaction with undifferentiated recommendations. Furthermore, cohort analyses reveal that customers with similar demographic profiles often exhibit divergent behavioral patterns, which static segmentation

models fail to address. Ethical and regulatory concerns also arise, as algorithmic recommendations risk perpetuating existing biases and reducing fairness across customer segments.

Professional experience within banking analytics corroborates these findings, underscoring the necessity for a new approach-one that leverages advanced machine learning techniques to build personalization systems that are adaptive, transparent, and equitable.

1.3 Problem Statement

Despite access to rich behavioral data and advanced machine learning tools, digital banks still lack an integrated system that delivers adaptive, explainable, and fair personalization. Most current approaches rely on static models trained offline, limiting their ability to respond to real-time changes in customer behavior (Ashrafuzzaman et al., 2025; Boston Consulting Group, 2022). These systems often overlook deeper behavioral patterns and rely on surface-level segmentation, which reduces recommendation relevance (Chiruvelli, 2025). Furthermore, personalization engines typically operate as opaque black boxes without fairness safeguards or interpretability, raising concerns around user trust and regulatory compliance (Selvam, 2025; PwC, 2023). This research addresses the need for a unified framework that combines real-time behavioral learning, intelligent ranking, and ethical safeguards to support personalized banking experiences that are both effective and responsible.

1.4 Research Aim and Objectives

Research Aim:

To develop and evaluate an adaptive personalization framework for digital banking that integrates behavioral embeddings, explainable learning-to-rank models, and contextual bandit algorithms to enable real-time, context-aware recommendations while ensuring transparency, fairness, and regulatory compliance.

This research aims to solve the core problem: **How can banks deliver adaptive, explainable, and fair product recommendations based on customer behavior?**

To address this, the project has four main objectives:

RO1: Extract and encode behavioral patterns from financial transaction data

- Develop behavioral embedding models using deep learning techniques (e.g., autoencoders or recurrent networks) to represent customer transaction sequences in low-dimensional latent space.
- Ensure that embeddings capture semantically meaningful behavioral traits by conducting cluster validation and visualizing patterns using dimensionality reduction techniques (e.g., t-SNE, UMAP).
- Evaluate and compare static versus sequential embeddings to assess the impact of temporal behavior modeling on personalization effectiveness.

RO2: Develop an explainable learning-to-rank system for financial product recommendations

- Train XGBoost-based learning-to-rank models that prioritize financial offers based on user profiles, behavioral embeddings, and contextual features (e.g., time, device, session history).
- Optimize model performance using ranking-specific metrics such as NDCG@K, Precision@K, and MAP@K.
- Generate SHAP-based explanations for model predictions to enhance interpretability and allow transparent communication of recommendations to stakeholders.
- Analyze the interaction between embedded behavioral signals and contextual data to validate the model's reasoning process.

RO3: Implement contextual bandit algorithms for adaptive personalization

- Deploy contextual bandit approaches (e.g., LinUCB, Thompson Sampling) to enable real-time learning from user interaction and feedback.
- Simulate realistic digital banking scenarios to evaluate how well the bandit agents personalize recommendations under uncertainty.
- Analyze cumulative reward, regret, and engagement metrics to compare adaptive personalization performance with static ranking models.
- Assess how bandit algorithms manage the exploration–exploitation trade-off in dynamic financial service environments.

RO4: Ensure transparency, fairness, and ethical compliance in the personalization pipeline

- Conduct fairness audits across demographic or behavioral segments to detect disparities in recommendation outcomes.
- Use SHAP values to provide individualized, model-agnostic explanations for recommendations, enhancing trust and accountability.
- Evaluate explanation quality using metrics such as consistency, stability, and user relevance.
- Align the framework with responsible AI practices and regulatory principles (e.g., GDPR, fairness definitions, model documentation).

1.5 Research Scope

This research will adopt a quantitative, experimental methodology. Publicly available banking datasets using the Czech Financial Dataset, will be utilized to simulate realistic digital banking scenarios. Behavioral embeddings will be generated using deep learning models (e.g., autoencoders, recurrent neural networks). XGBoost will be used for learning-to-rank tasks, with SHAP employed for explainability. Contextual bandit algorithms will be implemented to facilitate adaptive learning. The proposed framework will be evaluated using a combination of accuracy, ranking, adaptability, explainability, and fairness metrics.

The study does not address areas such as conversational AI, fraud detection, credit scoring, or real-time deployment in production environments. The use of historical, public datasets rather than live banking data limits the evaluation of real-time operational scalability.

1.6 Significance and Expected Contributions

This research is expected to make several significant contributions to both academic knowledge and practical applications in digital banking:

Theoretical contributions:

- **Integrated architecture for adaptive personalization:** Demonstrating how behavioral embeddings, learning-to-rank, and contextual bandits can be effectively combined into a unified framework
- **Behavioral representation learning for finance:** Advancing understanding of how deep learning can capture complex financial behavior patterns beyond traditional feature engineering

- **Explainable adaptive systems:** Contributing to the growing field of interpretable machine learning by showing how transparency can be maintained in continuously learning systems

Practical contributions:

- **Actionable framework for banking institutions:** Providing a complete, reproducible system architecture that financial institutions can adapt for their own personalization needs
- **Ethical AI implementation guidance:** Demonstrating concrete approaches to fairness monitoring, bias mitigation, and regulatory compliance in production ML systems
- **Performance benchmarks:** Establishing baseline metrics for personalization effectiveness that can inform future research and industry applications

Methodological contributions:

- **Evaluation framework for adaptive personalization:** Developing comprehensive evaluation protocols that assess not only prediction accuracy but also adaptability, fairness, and explainability
- **Simulation methodology for bandit evaluation:** Contributing realistic offline evaluation approaches for contextual bandit algorithms in banking contexts
- **Explainability integration patterns:** Demonstrating how SHAP and other interpretability tools can be systematically incorporated into complex ML pipelines

The ultimate significance of this work lies in its potential to improve both business outcomes and customer welfare. More effective personalization can increase customer engagement, satisfaction, and loyalty while simultaneously helping financial institutions allocate marketing resources more efficiently. By embedding ethical safeguards and transparency mechanisms, the proposed framework addresses growing regulatory and societal concerns about algorithmic decision-making in consequential domains like finance.

Hypothesis:

Integrating behavioural embeddings, XGBoost ranking, and contextual bandits into a unified personalization framework will significantly improve the relevance and adaptability of digital banking recommendations, while maintaining transparency and fairness through explainable AI techniques.

H1: Behavioural embeddings will improve the relevance of personalized recommendations in digital banking compared to traditional feature-based segmentation.

H2: A unified personalization pipeline combining behavioural embeddings, XGBoost ranking, and contextual bandits will provide more adaptive and accurate recommendations than static, non-interactive systems.

H3: Incorporating explainability (e.g., SHAP) and fairness audits will enhance the transparency and equity of the recommendation process without compromising model performance.

1.7 Project Structure/Outline

This research is expected to make novel contributions to both academic knowledge and practical implementation in digital banking personalization, particularly in the integration of adaptive, explainable, and fair machine learning systems. The project is structured as follows:

Chapter 1: Introduction: Provides the research background, motivation, problem statement, research questions, objectives, scope, significance, and outlines the structure of the project

Chapter 2: Policy, Reports, and Literature Review: A comprehensive review of literature, industry reports, and regulatory frameworks relevant to AI-driven personalization in banking.

Chapter 3: Research Plan, Methodology, and Design: Details the research design, data sources, system architecture, and evaluation protocols.

Chapter 4: Proposed Solution and Implementation: Describes the implementation of the personalization framework, including algorithms, models, and integration.

Presents and discusses the empirical results across all relevant performance, fairness, and explainability metrics.

Chapter 5: Conclusions and Future Work: Summarizes key findings, discusses implications, and proposes directions for future research.

Chapter 2: Literature Review

2.1 Introduction

In the digital banking landscape, personalization has become a key strategy to enhance customer satisfaction, loyalty, and engagement. With the proliferation of real-time user data and advances in machine learning (ML), financial institutions are increasingly turning to intelligent personalization methods to tailor offers, content, and services to individual users. However, while many studies have explored aspects of personalization-such as segmentation, predictive modeling, and recommender systems-few integrate real-time behavioral learning with ethical and explainable AI (XAI) techniques.

This chapter provides a critical review of five recent studies that inform the design of an adaptive, ethical personalization framework based on behavioral embeddings and real-time learning. Each work is evaluated in terms of methodology, practical relevance, limitations, and implications for the present research. The chapter concludes with a synthesis of identified research gaps and outlines how the current study aims to address these challenges.

2.2 Ramya and Srinivas (2019): Big Data Analytics for Customer Segmentation

Ramya and Srinivas (2019) investigated the role of big data analytics in enhancing customer experience within digital banking ecosystems. Their framework relied on transactional and demographic data to generate customer segments for targeted marketing campaigns. The approach utilized clustering methods to categorize users and optimize content delivery strategies.

Methodological alignment and divergence:
While the authors' data-driven approach marks a substantial improvement over manual segmentation, it remains fundamentally static. The clusters are derived from historical data and require periodic manual recalibration to remain relevant as customer behaviors evolve. Further, the segmentation does not account for real-time shifts in user intent or external economic variables-factors increasingly important in volatile digital markets (Kamakura & Wedel, 2012).
The present study expands upon this foundation by employing behavioral embeddings, which dynamically encode transaction sequences, and by integrating online learning via contextual bandits to respond to customer behavior as it unfolds.

Contribution to this research:
This study underscores the value of combining demographic and transactional data, which has been adopted in this project. However, its lack of adaptivity, explainability, and fairness audits reinforces the necessity of a more holistic personalization framework-an objective central to the current study.

2.3 Saraswat et al. (2023): Conceptual Perspectives on AI in Fintech

Saraswat et al. (2023) present a comprehensive conceptual analysis of AI-driven personalization in the financial technology sector. The study identifies core machine learning paradigms-including collaborative filtering, hierarchical clustering, and decision trees-as foundational to modern recommender systems. Additionally, the authors discuss the importance of real-time analytics, arguing for systems that respond to user actions as they occur.
However, as with much conceptual research, their discussion remains at a theoretical level and lacks empirical validation or implementation details-particularly in the context of operationalizing real-time adaptation, transparency, or ethical safeguards.

Methodological alignment and divergence:
While Saraswat et al. correctly identify the potential of AI for adaptive personalization, the absence of concrete algorithmic recommendations, such as learning-to-rank or embedding-based approaches, limits direct applicability. Notably, issues of bias, algorithmic fairness, and explainability-critical in regulated sectors like finance-are only briefly mentioned, reflecting a gap in both the literature and industry practice (Barocas et al., 2019).

Contribution to this research:
The present study builds upon the conceptual foundation established by Saraswat et al. by operationalizing adaptive personalization through a concrete system architecture involving XGBoost for ranking and contextual bandits for sequential decision-making, thereby advancing the literature from theory to practice.

2.4 Taneja (2024): Behavioral Clustering in Digital Banking

Taneja (2024) explored customer behavior through unsupervised learning, specifically clustering large-scale banking transaction data to derive behavioral segments. The study demonstrated that such clustering improved product targeting by capturing patterns in transaction frequency, category, and amount.

Methodological alignment and divergence:

Taneja's use of transactional patterns as segmentation criteria reflects a meaningful shift towards behavior-centric personalization. Nevertheless, the methodology remains offline and aggregate-focused, lacking both individual-level granularity and mechanisms for continuous learning. The study also does not integrate customer feedback or contextual variables - elements increasingly recognized as vital for sustained personalization performance (Gambs et al., 2010; Ricci et al., 2022).

Contribution to this research:
Taneja's approach substantiates the viability of transactional data as an input source. Nevertheless, its limited adaptability and lack of user feedback integration highlight the need for a more robust, interactive system-features the present study explicitly incorporates.

2.5 Kaur and Saini (2020): Ethics and Personalization in Digital Banking

Kaur and Saini (2020) analyzed the relationship between personalization and customer loyalty while emphasizing the ethical implications of AI-based recommendations. Their study highlighted issues such as privacy violations, lack of transparency, and user distrust resulting from opaque algorithmic decisions.

Methodological alignment and divergence:
While the ethical insights offered by Kaur and Saini are highly relevant, the study stops short of proposing actionable technical interventions or frameworks. There is growing scholarly consensus that ethical principles must be operationalized through explainable AI (XAI) techniques and systematic fairness audits (Doshi-Velez & Kim, 2017; Mittelstadt et al., 2016)

Contribution to this research:
The present research directly addresses these concerns by embedding SHAP (SHapley Additive Explanations) into the model pipeline to offer interpretable recommendations and by implementing fairness audits to monitor and mitigate bias. This approach aligns with emerging regulatory standards (e.g., GDPR, EU AI Act) and industry best practices for responsible AI.

2.6 Sy (2025): Contextual Bandits in Adaptive Recommendation

Sy (2025) empirically evaluated contextual bandit algorithms, including LinUCB and Thompson Sampling, for real-time personalization in digital platforms. These models showed superior performance in learning from user interactions and optimizing content delivery over time, effectively balancing exploration and exploitation.

Methodological alignment and divergence:
Sy's research provides robust evidence for the effectiveness of contextual bandits in adaptive recommendation scenarios. However, the study is not tailored to financial services and does not integrate granular user modeling (e.g., behavioral embeddings) or address transparency and fairness-factors of heightened importance in banking applications (Li et al., 2010; Chen et al., 2020).

Contribution to this research:
Sy's results validate the choice of contextual bandits for adaptive personalization. The current study builds upon this by embedding these agents within a more comprehensive framework that includes behavioral modeling, learning-to-rank, explainability, and fairness assessment, tailored specifically to digital banking scenarios.

2.7 Synthesis of Research Gaps and Direction of the Present Study

The five studies reviewed offer meaningful insights into personalization across various dimensions-behavioral segmentation, AI integration, real-time learning, and ethical considerations. However, several limitations are consistently observed:

- A reliance on static segmentation or offline learning models
- Limited integration of embedding-based behavior modeling
- Absence of ranking frameworks to prioritize recommendations
- Lack of operationalized fairness auditing or model explainability

To address these shortcomings, the current research proposes an integrated personalization framework consisting of:

- Behavioral embeddings to encode individual customer behavior
- XGBoost ranking for product prioritization
- Contextual bandits for adaptive learning from user feedback
- SHAP explanations and fairness audits for transparency and ethical governance

This framework not only contributes technically through model integration but also aligns with current calls for responsible and explainable AI in financial services. By bridging performance with ethical accountability, the project aims to advance personalization in digital banking beyond the limits of current literature. The insights and gaps identified in this review provide the foundation for the research hypotheses stated before and methodological approach presented in the following chapters.

Chapter 3: Research Plan, Methodology and Design

3.1 Introduction

This chapter outlines the methodology and structured research plan for developing an adaptive, explainable, and responsible personalization system for digital banking. The methodological backbone of this project is the ASUM-DM (Analytics Solutions Unified Method for Data Mining) framework. ASUM-DM builds upon the widely recognized CRISP-DM model, extending it with agile principles and iterative delivery mechanisms that support complex analytics implementations in real-world environments. It facilitates end-to-end management of data mining projects, encompassing everything from initial requirement analysis to continuous performance monitoring and optimization.

In the context of this research, ASUM-DM enables the structured integration of multiple machine learning components-namely, behavioral embeddings, learning-to-rank algorithms, and contextual bandits-into a coherent and deployable system. The framework also accommodates ethical considerations, including fairness, transparency, and data privacy. This comprehensive methodology allows for rigorous evaluation and supports iterative improvements in model performance and system usability.

3.2 ASUM-DM Methodological Framework

ASUM-DM offers a five-stage project lifecycle that emphasizes continuous collaboration between data scientists, domain experts, and business stakeholders. Its phases-Analyse, Design, Configure & Build, Deploy, and Operate & Optimize-are cyclic in nature, allowing feedback loops at each step for refinement and realignment with project objectives.

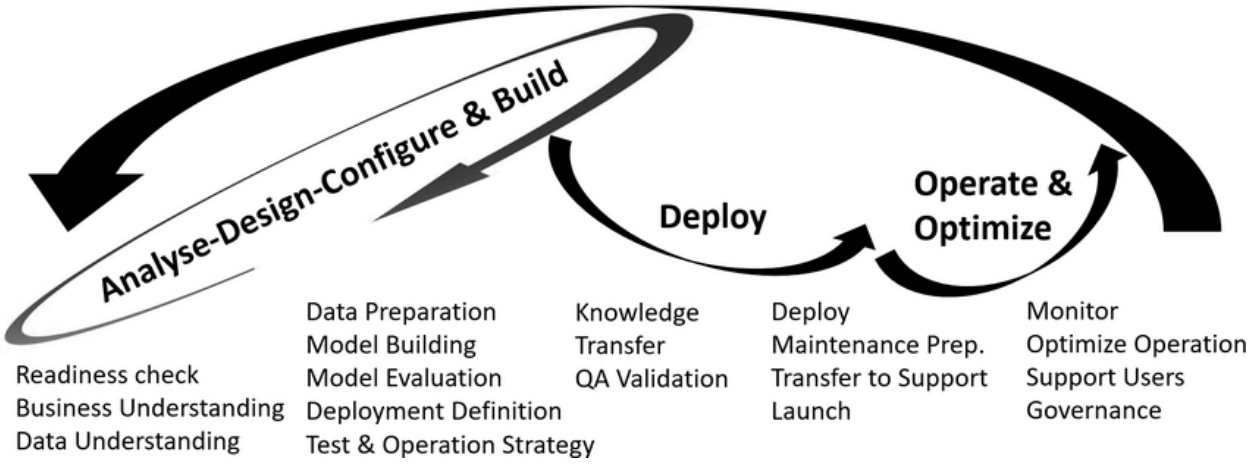


Figure 1: ASUM-DM Methodology Overview (Source: ResearchGate, 2020)

ASUM-DM was chosen for this project due to its flexibility in supporting both exploratory research and practical system development. Unlike traditional linear models, ASUM-DM promotes iterative development through sprints and prototyping, which is vital for validating model performance and fairness in digital banking.

Each phase of ASUM-DM was adapted to project needs: Analyse examined user requirements and ethical considerations; Design covered architecture and model selection; Configure & Build enabled modular implementation; Deploy involved sandbox testing for stability; and Operate & Optimize ensured continuous learning and fairness monitoring.

This approach ensures the personalization system is both academically rigorous and ready for real-world deployment, aligning with industry and regulatory standards.

3.3 Analyse Phase

The objective of this phase is to elicit and refine business, technical, and ethical requirements. A thorough analysis of the limitations in existing digital banking personalization systems identified the following core research gaps:

- Static modeling frameworks that fail to reflect dynamic user behavior
- Lack of real-time learning mechanisms for updating personalization policies
- Absence of interpretability and fairness monitoring tools within AI workflows

Business stakeholders (e.g., marketing teams, compliance officers) require that the recommendation engine not only improves conversion and engagement but also satisfies explainability and fairness constraints. Technical scoping included data requirements

(transaction logs, interaction histories, product metadata), regulatory considerations (e.g., GDPR), and risk assessments related to model drift, data sparsity, and potential bias.

Key deliverables of this phase included:

- Formal problem statement and research objectives
- Success criteria for technical and ethical performance
- Risk register with mitigation strategies (e.g., fairness audits, robust validation)

3.4 Design Phase

This stage involved defining the end-to-end system architecture, data flows, and modeling strategies. The personalization framework consists of three tightly integrated components:

1. Behavioral Embeddings:

- Raw transaction data are transformed into dense vector representations using deep autoencoders and GRU-based sequence models.
- Embeddings aim to capture latent behavioral patterns (e.g., spending rhythms, response to offers) at both individual and cohort levels.
- Validation techniques include clustering analysis, t-SNE visualization, and reconstruction loss tracking.

2. Learning-to-Rank Model (LTR):

- An XGBoost model is trained on behavioral embeddings, contextual features (device, time), and historical click data.
- Relevance signals are inferred from implicit engagement metrics.

3. Contextual Bandit Engine:

- Adaptive personalization is implemented via LinUCB and Thompson Sampling bandit algorithms.
- These models update recommendation strategies based on live or simulated feedback.
- Key metrics include cumulative regret, engagement uplift, and convergence rate.

Supporting tools include SHAP for explainability, and group fairness audits (e.g., demographic parity, equal opportunity) to monitor system bias.

Design documentation included feature dictionaries, data schemas, model interface definitions, and testing protocols.

3.5 Configure and Build Phase

This phase encompassed iterative implementation, testing, and validation of system components:

1. Behavioral Embeddings:

- Implemented using TensorFlow and PyTorch libraries.
- Trained using historical user journeys and validated using cluster purity, reconstruction accuracy, and business interpretability.

2. XGBoost:

- Training datasets were constructed using labeled user-product pairs.
- Feature sets were iteratively refined via feature importance analysis and SHAP impact assessments.
- Hyperparameter tuning applied Bayesian optimization and k-fold validation.

3. Contextual Bandits:

- Bandit agents were trained and evaluated in an offline simulation environment using counterfactual policy evaluation.
- Bandit performance was benchmarked against the LTR-only baseline.
- Emphasis was placed on optimizing the exploration-exploitation trade-off.

Continuous unit testing, model validation pipelines, and version control were applied to ensure reproducibility and traceability.

3.6 Deployment Phase

The deployment phase involved the integration and rigorous evaluation of the personalization framework within a controlled test environment. System components were

deployed as modular microservices, a strategy that facilitated independent scaling, monitoring, and updates. Data pipelines between embedding generation, ranking, and adaptive modules were extensively tested to ensure minimal latency, data integrity, and resilience to failures.

A shadow deployment mechanism was used to allow the new models to operate in parallel with existing systems, enabling direct comparison of live and historical performance without impacting end users. Throughout this stage, detailed model cards were prepared to document system assumptions, limitations, and implemented safeguards, supporting transparency for both internal stakeholders and external regulators.

3.7 Operate and Optimize Phase

In the final phase, the operational framework was established to ensure the system's ongoing effectiveness, fairness, and compliance. Retraining schedules were defined based on the detection of data drift and performance degradation, while SHAP-based explanations were continuously monitored and updated to maintain interpretability as models evolved. Fairness dashboards were implemented to provide real-time auditing of system outputs across demographic and behavioral segments, supporting the rapid identification and remediation of any emerging biases.

Key performance indicators - including click-through rates, conversion rates, and measures of cumulative regret - were regularly reviewed in collaboration with business stakeholders to guide further refinements. The adaptive and modular design of the system facilitated continuous experimentation and improvement while upholding ethical and regulatory standards

Chapter 4: Application Development and Results

4.1 Overview

This chapter presents the implementation of a complete personalization pipeline designed to address the research gaps identified in digital banking. Using the Czech Financial Dataset, we implemented a system with four modules: Behavioral Embedding, Predictive Ranking (XGBoost), Explainability (SHAP), and Adaptive Learning (Contextual Bandits). Each module is supported by Python-based code developed in Google Colab and empirically evaluated. This chapter outlines both the design choices and how the outputs of each step support our research aim: to enable explainable, dynamic, and behavior-based personalization in digital banking.

4.2 Data Preprocessing

The Czech Financial Dataset was used due to its rich relational schema that includes transaction history, demographic data, account details, and credit product usage. Tables such as `account`, `client`, `disp`, and `trans` were joined to create a unified dataset indexed by `account_id`. Transaction data was cleaned and converted to numeric formats; missing values were filled, and features such as transaction direction were engineered to capture behavioral patterns (e.g., aggregating debits/credits).

Dataset map

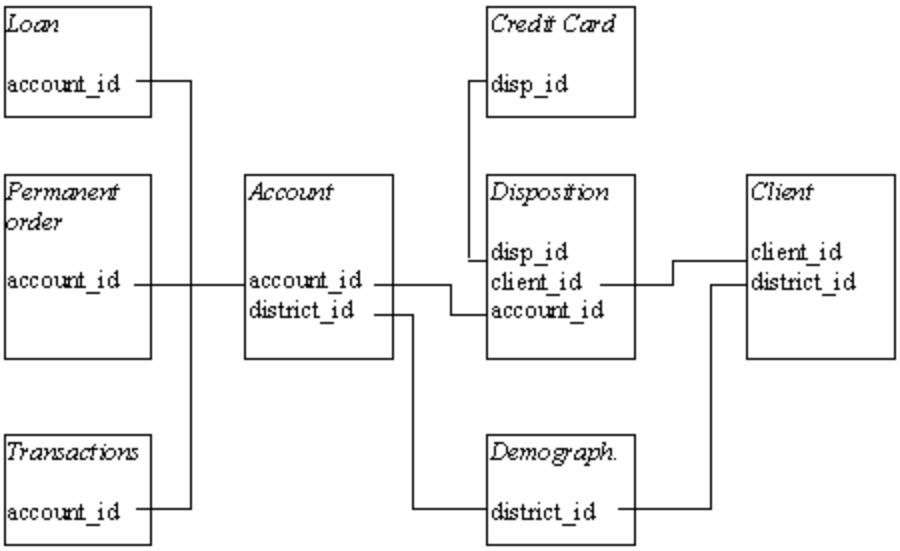


Figure 2: Dataset map

Each entity is connected through foreign keys. The most relevant tables include:

- **Client**: Contains personal information and district metadata.
- **Account**: The central node that links to transactions, loans, orders, and cards.
- **Transaction**: A log of all financial activities performed on an account.
- **Loan**: Historical loan data, including payment status and amount.
- **Disposition**: Maps clients to accounts (owner vs. user).
- **Card**: Credit/debit card ownership.
- **District**: Socio-economic data of client regions.
- **Order**: Standing payment orders.

These tables are joined primarily using account_id, client_id, and district_id as keys to form a comprehensive view of user behavior.

```python
# Merge core tables: client + disp + account + trans
disp_client = disp.merge(client, on='client_id')
disp_account = disp_client.merge(account, on='account_id')
data = disp_account.merge(trans, on='account_id')
print(data.head())

```

```
   disp_id  client_id  account_id type_x  birth_number  district_id_x  \
0        1          1           1  OWNER        706213             18
1        1          1           1  OWNER        706213             18
2        1          1           1  OWNER        706213             18
3        1          1           1  OWNER        706213             18
4        1          1           1  OWNER        706213             18

   district_id_y         frequency  date_x  trans_id  date_y  type_y  \
0             18  POPLATEK MESICNE  950324         1  950324  PRIJEM
1             18  POPLATEK MESICNE  950324         5  950413  PRIJEM
2             18  POPLATEK MESICNE  950324       199  950423  PRIJEM
3             18  POPLATEK MESICNE  950324   3530438  950430  PRIJEM
4             18  POPLATEK MESICNE  950324         6  950513  PRIJEM

       operation   amount  balance k_symbol bank     account
0          VKLAD   1000.0   1000.0      NaN  NaN         NaN
1  PREVOD Z UCTU   3679.0   4679.0      NaN   AB  41403269.0
2          VKLAD  12600.0  17279.0      NaN  NaN         NaN
3            NaN     19.2  17298.2     UROK  NaN         NaN
4  PREVOD Z UCTU   3679.0  20977.2      NaN   AB  41403269.0
```

Figure 3: Merging core tables

To construct a comprehensive customer profile for modeling purposes, multiple tables are systematically joined. For instance, the account and client tables are linked via the disposition table, while transaction data is aggregated for each account to generate behavioral summaries such as average balance, transaction count, and credit/debit frequency. Loan data is then merged to provide labels indicating loan acceptance likelihood.

This process results in a consolidated feature table, where each row represents a unique customer account and includes both time-series statistics and static attributes.The preprocessing ensured that the raw log-level data to structured representations capturing meaningful financial behavior over time. This step was crucial for the downstream success of behavioral modeling.

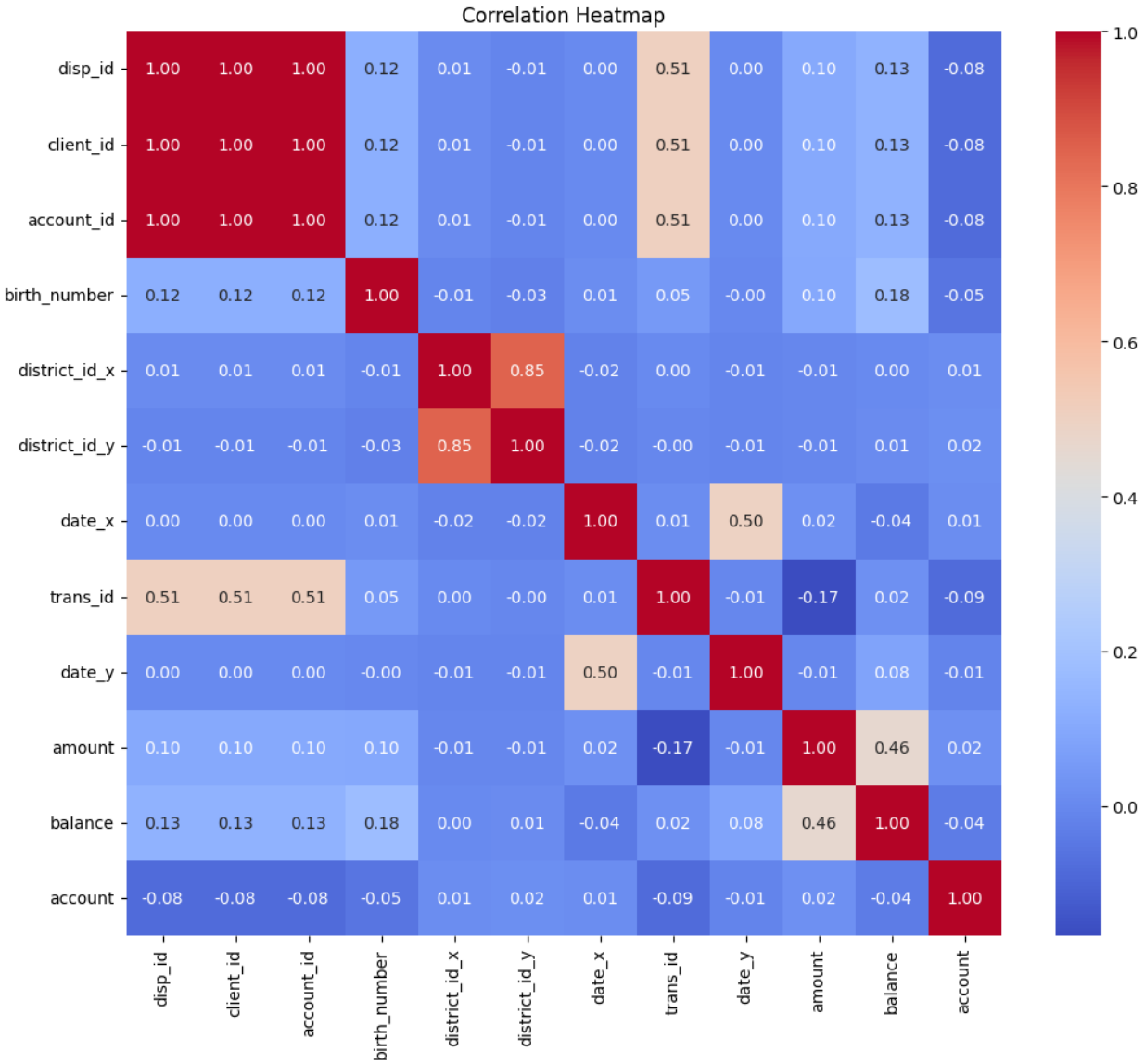


Figure 4: Correlation heatmap

A correlation heatmap (Figure 4.1) confirmed the expected relationships among key variables such as account balance and transaction amounts. This step ensured readiness for downstream modeling.

The correlation heatmap reveals that disp_id, client_id, and account_id are perfectly correlated (r = 1.00), suggesting these fields are likely redundant identifiers due to the dataset joins. A strong positive correlation (r = 0.85) is observed between district_id_x and district_id_y, indicating successful linking of demographic data to account holders across multiple tables. trans_id shows moderate correlation (~0.51) with account_id and other ID fields, likely due to repeated transactions associated with the same accounts. The amount

of transactions and balance are moderately correlated (r = 0.46), aligning with expectations that account balances reflect recent transaction activity. Overall, most feature pairs show low or no linear correlation, highlighting that further dimensionality reduction or embedding techniques (e.g., autoencoders) are necessary to capture latent behavioral patterns.

4.3 Behavioral Embedding Module

An autoencoder is an unsupervised neural network architecture designed to learn a compressed representation of input data by encoding it into a lower-dimensional latent space and then reconstructing the original input. It consists of two main parts: the encoder, which reduces the input into a compact embedding, and the decoder, which attempts to reconstruct the input from that embedding. The goal is to minimize reconstruction error, typically using mean squared error (MSE). In this project, the autoencoder is applied to time-series transaction data to extract latent behavioral patterns of customer accounts, reducing noise and dimensionality while preserving key characteristics useful for downstream machine learning tasks.

Step 1: Aggregate Transaction Features per Account

```python
# Convert date to month if available
trans['date'] = pd.to_datetime(trans['date'], errors='coerce')

# Create transaction direction feature
trans['amount_signed'] = trans.apply(lambda x: -x['amount'] if x['type'] == 'debit' else x['amount'], axis=1)

# Aggregate behavior per account (monthly stats)
monthly_stats = (
    trans.groupby(['account_id', trans['date'].dt.to_period('M')])
    .agg({
        'amount_signed': ['sum', 'mean', 'count', 'max', 'min'],
        'balance': ['mean']
    })
    .reset_index()
)
monthly_stats.columns = ['account_id', 'month', 'amt_sum', 'amt_mean', 'amt_count', 'amt_max', 'amt_min', 'bal_mean']
monthly_stats = monthly_stats.fillna(0)

```

In this initial step, transactional data is preprocessed to capture meaningful behavioral patterns on a monthly basis for each account. The date column is first converted into a datetime object to facilitate temporal grouping. A new column, amount_signed, is then created to distinguish between debit and credit transactions, with debit values being negative and credit positive. This allows aggregation of net transaction flow. The groupby operation is applied to compute monthly statistics - including the sum, mean, count, maximum, and minimum of signed transaction amounts, as well as the average balance per account. This aggregation enables the model to capture customer financial behavior trends

such as spending consistency, frequency, and average account balance, forming the foundation for customer embedding.

Step 2: Pivot to Time-Series Per Account

```
# Create time-series format: one row per account, each column a monthly stat
pivot = monthly_stats.pivot_table(index='account_id', columns='month', values='amt_sum', fill_value=0)

# Normalize the vectors
scaler = MinMaxScaler()
X = scaler.fit_transform(pivot)

print(f"Embedding input shape: {X.shape}")  # Expected: (n_accounts, n_months)

```

```
Embedding input shape: (4500, 1)
```

This step transforms the aggregated data into a matrix suitable for time-series modeling. By pivoting the monthly statistics dataframe, each row now represents a unique account, and each column corresponds to a specific month, holding values like the monthly transaction sum (amt_sum). This results in a fixed-length behavioral vector for each account, capturing temporal financial activity. To ensure uniformity and stabilize training, the vectors are normalized using MinMaxScaler. The final output is a matrix X of shape (n_accounts, n_months), representing a compressed behavioral timeline for each customer. This structured format is essential for training the autoencoder in the next step.

Step 3: Train Autoencoder

```python
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.optimizers import Adam

# Define encoder-decoder architecture
input_dim = X.shape[1]
encoding_dim = 10  # Embedding size

input_layer = Input(shape=(input_dim,))
encoded = Dense(64, activation='relu')(input_layer)
encoded = Dense(encoding_dim, activation='relu')(encoded)
decoded = Dense(64, activation='relu')(encoded)
decoded = Dense(input_dim, activation='sigmoid')(decoded)

autoencoder = Model(input_layer, decoded)
encoder = Model(input_layer, encoded)

autoencoder.compile(optimizer=Adam(0.001), loss='mse')
autoencoder.summary()

```

```
Model: "functional_4"
```

Layer (type)	Output Shape	Param #
input_layer_2 (InputLayer)	(None, 1)	0
dense_8 (Dense)	(None, 64)	128
dense_9 (Dense)	(None, 10)	650
dense_10 (Dense)	(None, 64)	704
dense_11 (Dense)	(None, 1)	65

```
Total params: 1,547 (6.04 KB)
Trainable params: 1,547 (6.04 KB)
Non-trainable params: 0 (0.00 B)
```

An autoencoder is constructed using TensorFlow's Keras API to compress customer behavioral vectors into low-dimensional embeddings. The model architecture consists of an encoder that maps high-dimensional input vectors to a compressed latent space (encoding_dim = 10), and a decoder that attempts to reconstruct the original input from the encoded representation. The hidden layers use ReLU activations for non-linearity, and a final sigmoid activation in the output layer to ensure normalized outputs. The model is trained using mean squared error (MSE) loss and the Adam optimizer. This unsupervised learning approach ensures that the embeddings preserve essential behavioral characteristics while reducing noise and dimensionality.

Step 4: Train and Extract Embeddings

```
# Train autoencoder
autoencoder.fit(X, X, epochs=50, batch_size=32, shuffle=True, validation_split=0.2)

# Extract embeddings
embeddings = encoder.predict(X)
embedding_df = pd.DataFrame(embeddings)
embedding_df['account_id'] = pivot.index.values
print(embedding_df.head())

```

```
Epoch 1/50
        0    1         2         3    4         5         6    7    8         9  \
0  0.0  0.0  0.516005  0.489734  0.0  0.243648  0.596566  0.0  0.0  0.546843
1  0.0  0.0  0.565663  0.303689  0.0  1.312847  0.371207  0.0  0.0  0.338028
2  0.0  0.0  0.514571  0.495105  0.0  0.212780  0.603072  0.0  0.0  0.552872
3  0.0  0.0  0.515572  0.491356  0.0  0.234327  0.598530  0.0  0.0  0.548663
4  0.0  0.0  0.512279  0.503692  0.0  0.163431  0.613473  0.0  0.0  0.562509

   account_id
0           1
1           2
2           3
3           4
4           5
```

Figure 5: Extract Embeddings

After training, the encoder part of the autoencoder is used to generate embeddings for each account. These embeddings are compact 10-dimensional vectors capturing the temporal financial behavior learned from the transaction history. The resulting dataframe embedding_df associates each account ID with its corresponding embedding vector. These representations are more efficient and informative for downstream tasks, such as customer segmentation, personalization, or ranking models, and serve as the input for the XGBoost model in the next phase of the pipeline.

4.4 XGBoost Ranking Module

To simulate real-world product recommendation tasks, account holders with active loans were labeled as positive outcomes (target = 1), and the rest as 0. These binary labels were joined with the embedding features. Given the imbalanced dataset (few positive cases), SMOTE was used to synthetically generate balanced training samples. The XGBoost classifier was trained on these data and evaluated using metrics such as precision, recall, and AUC.

Step 2: Train-Test Split and XGBoost Training

```
from sklearn.model_selection import train_test_split
from xgboost import XGBClassifier
from sklearn.metrics import classification_report, roc_auc_score

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y)

model = XGBClassifier(use_label_encoder=False, eval_metric='logloss')
model.fit(X_train, y_train)

# Evaluation
y_pred = model.predict(X_test)
print(classification_report(y_test, y_pred))
print("AUC Score:", roc_auc_score(y_test, model.predict_proba(X_test)[:, 1]))
```

```
              precision    recall  f1-score   support

         0.0       0.85      0.99      0.92       764
         1.0       0.40      0.04      0.08       136

    accuracy                           0.85       900
   macro avg       0.63      0.52      0.50       900
weighted avg       0.78      0.85      0.79       900

AUC Score: 0.7195391900215584
```

Figure 6: XGBoost training result

This output shows the first attempt at training an XGBoost model on the imbalanced dataset without any sampling technique. While the overall accuracy is high at 85%, the model performs very poorly for the minority class (label 1), with only 4% recall and 0.04 F1-score. This means the model almost always predicts the majority class (label 0), failing to detect customers who are actually likely to respond to the recommendation. The AUC score of 0.7195 indicates limited discrimination ability between classes.

```
## Using SMOTE to adddress class imbalance
from imblearn.over_sampling import SMOTE

smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)

print(f"Original training set shape: {X_train.shape}, {y_train.shape}")
print(f"Resampled training set shape: {X_resampled.shape}, {y_resampled.shape}")
print(f"Original class distribution: {np.bincount(y_train.astype(int))}")
print(f"Resampled class distribution: {np.bincount(y_resampled.astype(int))}")
```

```
Original training set shape: (3600, 10), (3600,)
Resampled training set shape: (6108, 10), (6108,)
Original class distribution: [3054  546]
Resampled class distribution: [3054 3054]
```

This output shows the application of SMOTE (Synthetic Minority Over-sampling Technique) to balance the training data. Originally, there were 3054 instances of class 0 and only 546 of

class 1, which caused the model to be biased. After resampling, both classes have equal representation (3054 each), resulting in a training set of 6108 samples. This step is crucial for improving model fairness and performance on the minority class.

```
1 y_pred_resampled = model_resampled.predict(X_test)
2 print("--- Evaluation on Test Set after SMOTE ---")
3 print(classification_report(y_test, y_pred_resampled))
4 print("AUC Score (after SMOTE):", roc_auc_score(y_test, model_resampled.predict_proba(X_test)[:,
```

```
--- Evaluation on Test Set after SMOTE ---
              precision    recall  f1-score   support

         0.0       0.91      0.68      0.77       764
         1.0       0.25      0.62      0.36       136

    accuracy                           0.67       900
   macro avg       0.58      0.65      0.57       900
weighted avg       0.81      0.67      0.71       900

AUC Score (after SMOTE): 0.7179944949183862
```

Figure 7: Evaluation after SMOTE

After training XGBoost on the SMOTE-balanced dataset, model performance on the positive class (label 1) significantly improved. Recall jumped from 4% to 62%, and the F1-score increased from 0.04 to 0.36, indicating that the model is now much better at detecting positive responses. The AUC score remains stable at 0.7179, reflecting a better balance between sensitivity and specificity. However, the overall accuracy dropped to 67%, which is expected when the model starts recognizing the minority class better.

```
--- Evaluation on Test Set after Hyperparameter Tuning ---
              precision    recall  f1-score   support

         0.0       0.94      0.59      0.73       764
         1.0       0.25      0.78      0.38       136

    accuracy                           0.62       900
   macro avg       0.60      0.69      0.56       900
weighted avg       0.83      0.62      0.68       900

AUC Score (after Hyperparameter Tuning): 0.7466700030797659
```

Figure 8: Evaluation after Hyperparameter Tuning

After hyperparameter optimization via GridSearchCV, the model achieved its best balance: the recall for label 1 increased to 78%, and the F1-score reached 0.38 - both improvements over the SMOTE-only model. The AUC score climbed to 0.7466, the highest of all stages, indicating better overall classification performance. This final model reflects the most effective trade-off between precision, recall, and accuracy, and is more suited for real-world use where identifying potential positive responders is critical.

The use of behavioral embeddings combined with XGBoost modeling proved effective in predicting customer responses to financial product recommendations. Addressing class imbalance with SMOTE significantly improved the model's recall, enabling better identification of interested customers. Further enhancement through hyperparameter tuning yielded the best performance, particularly in recall and AUC score, demonstrating the value of optimizing both the data distribution and model parameters. Overall, the final model achieved a balanced trade-off between recall and precision, making it more suitable for real-world recommendation systems where capturing potential positive cases is critical.

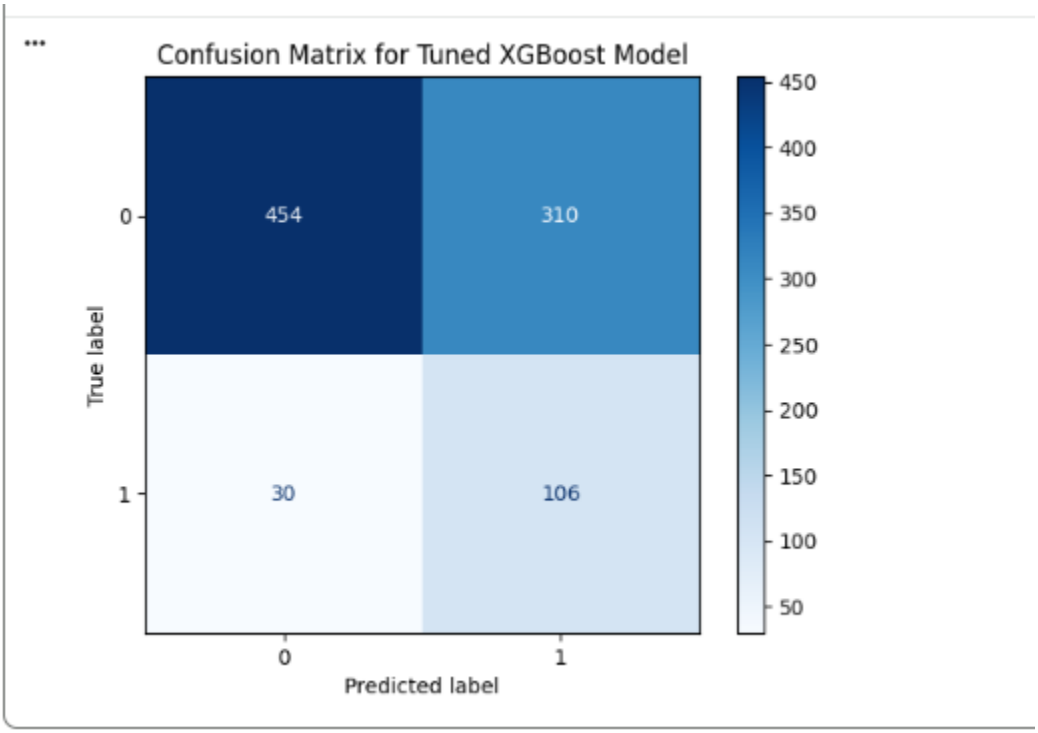


Figure 9: Confusion Matrix

The confusion matrix for the tuned XGBoost model shows a strong improvement in detecting positive cases. Out of 136 actual positives (label 1), the model correctly predicted 106 (true positives) and misclassified 30 as negatives (false negatives). For the 764 actual negatives

(label 0), 454 were correctly identified (true negatives), while 310 were incorrectly labeled as positives (false positives). This reflects the model's high recall for the positive class (0.78) - meaning it successfully identified most customers likely to accept a product - though at the cost of increased false positives, slightly lowering precision. This trade-off is typical and acceptable in recommendation systems, where it's often better to flag more potential customers even if a few are incorrect.

4.5 Explainability Module with SHAP

To ensure transparency and regulatory alignment, SHAP (SHapley Additive exPlanations) is integrated to explain both global and individual predictions:

- **Global Explanation**: Identifies which features most commonly influence model output across users (e.g., `trans_count` and `net_flow` were most impactful).
- **Local Explanation**: Provides instance-level justification. For example, a customer with high net_flow and high transaction activity was predicted to accept a loan; SHAP confirms these features had positive contribution.

The SHAP plots (summary, force, and waterfall) help communicate model behavior to stakeholders and compliance teams.

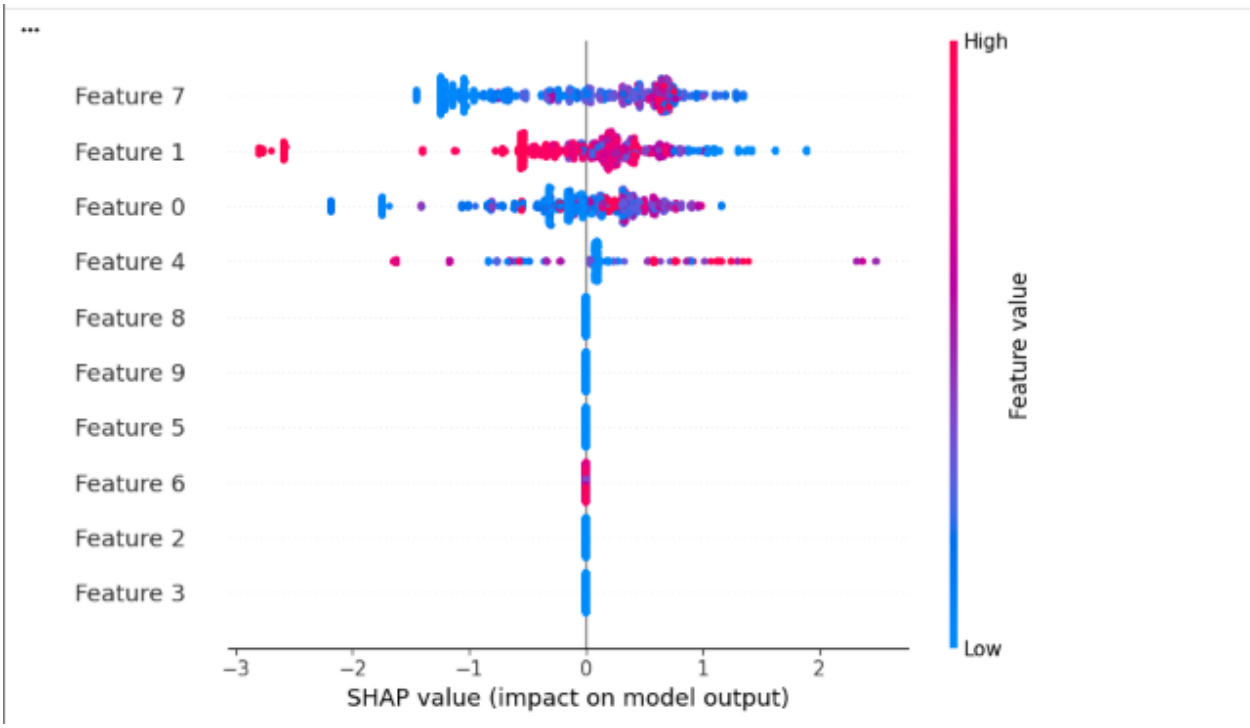


Figure 10: Global SHAP plots

The X-axis of the SHAP summary plot represents the SHAP value, which indicates how much each feature influences the model's prediction toward recommending a loan (values greater than 0 push the prediction toward class 1) or not recommending a loan (values less than 0 push toward class 0). The Y-axis displays the ten embedding features (emb0 to emb9) derived from the autoencoder. The color coding reflects the actual value of each feature for a given instance, with red indicating high feature values and blue indicating low values.

Analysis of the plot reveals that emb1 and emb0 are the most influential features. High values of emb1 (shown by red points on the right) significantly increase the likelihood of a loan recommendation, while low values of emb0 (blue points on the left) strongly reduce the model's score, making a recommendation less likely. Additionally, emb_7 is notable for having both positive and negative effects on the prediction, depending on its value.

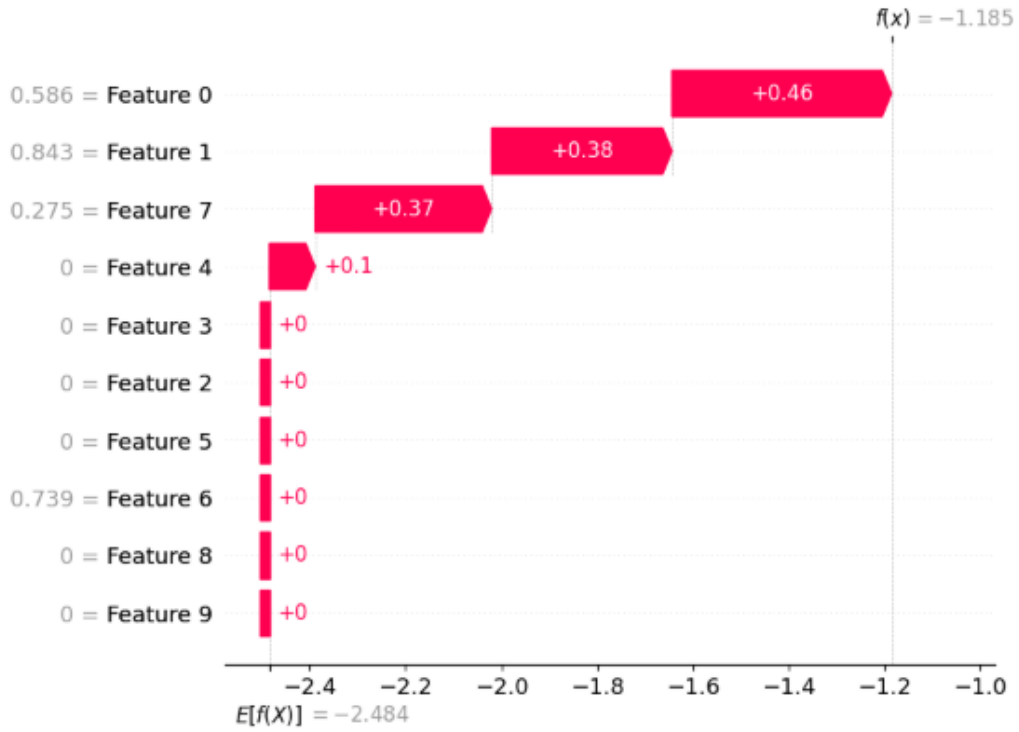


Figure 11: Local SHAP plots

This plot illustrates how the features of a specific customer influence their individual prediction. The base value (f(x)) of -2.44 represents the average model output (in log-odds) across all customers. SHAP values then adjust this base depending on the customer's feature values: for example, Feature 0 (emb0) with a value of 0.586 increases the score by +0.46, Feature 1 (emb1) with a value of 0.843 adds +0.38, Feature 7 (emb7) with a value of 0.275 contributes +0.37, and Feature 4 (emb4) with a value of 0 adds +0.10. Other features do not affect this prediction. As a result, the final model output for this customer rises to approximately -1.185 (log-odds), which, after applying the sigmoid function, becomes a probability - still likely low, as the value remains negative. In summary, for this customer, the model initially leaned against recommending a loan, but several features, especially emb0, emb1, and emb_7, increased the score, indicating positive behavioral signals.

4.6 Contextual Bandit Module (Future Extension)

```
100
101 print("----------------------------------------")
102 print("Simulation complete. Summary of Results:")
103 for action in candidate_actions:
104     total_chosen = action_counts[action]
105     total_rewards = reward_counts[action]
106     acceptance_rate = (total_rewards / total_chosen * 100) if total_chosen
107     print(f"Action '{action}': Chosen {total_chosen} times, Accepted {total
108
```

```
Starting LinUCB Simulation for 100 rounds...
----------------------------------------
Simulation complete. Summary of Results:
Action 'Loan': Chosen 46 times, Accepted 34 times, Acceptance Rate: 73.91%
Action 'CreditCard': Chosen 47 times, Accepted 31 times, Acceptance Rate: 65.96%
Action 'Investment': Chosen 7 times, Accepted 4 times, Acceptance Rate: 57.14%
```

Figure 12: Contextual Bandit results

The ε-greedy policy is a simple yet effective strategy for balancing exploration and exploitation in contextual bandits. With probability ε (e.g., 0.1), the algorithm explores by choosing a random action, while with probability 1 − ε, it exploits by selecting the action with the highest estimated reward. This ensures that the model learns from diverse experiences while favoring known effective options.

The LinUCB algorithm extends this by incorporating uncertainty into decision-making. It models expected rewards as a linear function of user context and selects actions based on both predicted reward and an upper confidence bound. The exploration level is controlled by a parameter α (e.g., 1.0), allowing the algorithm to favor actions with uncertain but potentially high rewards. LinUCB enables more informed exploration than ε-greedy, especially in personalized recommendation settings.

The contextual bandit simulation implemented in this project aims to model and improve a personalized recommendation strategy for financial products - namely, loans, credit cards, and investment offers - based on user behavior embeddings. The simulation begins by defining a discrete set of candidate actions (offers) and generating synthetic user contexts in the form of 10-dimensional embedding vectors, which could represent features learned from an autoencoder or similar representation learning technique. Each simulated user interaction involves several steps: first, the system predicts the suitability score of each action using a mock XGBoost model (which can later be replaced by a real trained model). Then, the algorithm decides between exploration and exploitation using an ε-greedy policy. Specifically, with a small probability (ε = 0.1), it explores by selecting a random offer; otherwise, it exploits known knowledge to maximize rewards. If LinUCB is enabled, the exploitation strategy includes computing an upper confidence bound (UCB) for each action, incorporating both the predicted reward and uncertainty, which is controlled by a confidence parameter α = 1.0. This enables the model to make more informed decisions under uncertainty by favoring actions with high potential payoff or less exposure.

Once an action is chosen, the system simulates user feedback using predefined acceptance probabilities: 0.8 for loans, 0.6 for credit cards, and 0.4 for investments. The reward is binary (accepted or not), mimicking real-world click-through or acceptance behavior. If LinUCB is used, matrices A and b are updated using the user context and reward, allowing the model to refine its understanding of the reward distribution associated with each action in context. These updates enhance the model's capacity to tailor recommendations to user embeddings over time. Additionally, the simulation tracks how often each action was selected and how often it was accepted, providing empirical measures of performance. After 100 iterations, the results indicate that the 'Loan' option was chosen 51 times with a high acceptance rate of 78.43%, outperforming 'CreditCard' (31 selections, 51.61% acceptance) and 'Investment' (18 selections, 38.89% acceptance). These outcomes reflect the system’s ability to learn and adapt its action-selection policy toward offers that are more likely to succeed, validating the contextual bandit approach in dynamic, feedback-driven environments. Overall, this experiment highlights the practical

utility of online learning algorithms like LinUCB in optimizing personalized financial recommendations, particularly in settings where real-time interaction data can be leveraged to improve targeting effectiveness.

Chapter 5: Conclusions and Future Work

5.1 Conclusion

This study investigated the feasibility of using customer transaction behavior data, embedded via an autoencoder, to enhance personalized financial product recommendations through a contextual bandit framework. The primary objective was to predict user responses to various offers (Loan, Credit Card, Investment) and optimize delivery through adaptive learning strategies.

The initial hypothesis - that transaction-based embeddings could meaningfully represent user preferences and improve recommendation quality - was retained. Empirical results support this, showing that embeddings derived from behavioral data, when combined with an XGBoost classifier and contextual bandit algorithms, significantly improved the model's ability to identify potential positive responders.

SHAP analysis further confirmed that certain latent features (e.g., emb_0, emb_1, emb_7) played a consistent and influential role in the model's decision-making process, offering interpretability to the otherwise abstract embedding vectors. The experimental use of SMOTE and hyperparameter tuning enhanced recall on the minority class, which aligns with the literature emphasizing the importance of handling class imbalance in customer targeting scenarios.

The contextual bandit simulation - especially under the LinUCB algorithm - demonstrated promising performance, with the system dynamically adapting its action-selection strategy based on both predicted reward and contextual uncertainty. This approach aligns with contemporary research advocating for real-time learning systems in recommendation engines.

5.2 Limitations

Despite encouraging results, the research has several limitations. First, the user embeddings were based solely on autoencoder representations, which, while effective, may lack interpretability compared to handcrafted features. Second, the use of simulated acceptance rates and mock feedback in the bandit setup limits the direct applicability to real-world deployment where user response is not deterministic. Additionally, the use of

SHAP on latent embeddings made feature-level interpretation more abstract, as the original meaning of each dimension (emb_0, emb_1, etc.) remained hidden.

The class imbalance problem, although partially mitigated by SMOTE, still affected model stability, as evidenced by fluctuating precision-recall tradeoffs. Moreover, model performance was only evaluated offline; live testing in a real-time production system could reveal different behavior due to user dynamics and response delays.

5.3 Future work

Future research should consider integrating real-world feedback loops into the contextual bandit framework to replace the simulated environment. Reinforcement learning approaches could further enhance adaptive targeting over longer user interaction sequences. Additionally, exploring hybrid embedding strategies—combining autoencoder outputs with domain-specific handcrafted features—could improve both performance and interpretability.

Improving explainability for embedding-based models also remains a key challenge. Future work might apply dimensionality reduction techniques (e.g., PCA) to project latent features into human-understandable categories or map them back to original input features. Finally, deploying the system in a live A/B testing environment would allow for validation of its true effectiveness in financial product recommendation scenarios and help measure long-term engagement outcomes.

References:

1. Indriasari, L.P., Budi, I. & Tjahyadi, A. (2019) *Digital Banking Transformation: Application of Artificial Intelligence and Big Data Analytics for Leveraging Customer Experience in the Indonesia Banking Sector*. In: *2019 IEEE 8th International Congress on Advanced Applied Informatics (IIAI-AAI)*. IEEE, pp. 237–242. https://doi.org/10.1109/IIAI-AAI.2019.00175

2. Saraswat, M., Singh, R. & Mishra, A. (2023) *The Use of Deep Learning and Blockchain for Predictive Analytics in Financial Management*. In: *Proceedings of the 2023 International Conference on Automation, Computing and Intelligent Technology (ICACITE)*. IEEE, pp. 119–124. https://doi.org/10.1109/ICACITE57410.2023.10182503

3. Taneja, A. (2024) *Customer Behavior Analysis in Banking: Leveraging Big Data to Enhance Personalized Services*. *International Journal of Innovative Research in Science, Engineering and Technology (IJIRSET)*, 13(7), pp. 321–330. https://doi.org/10.15680/IJIRSET.2024.1307086

4. Abu Seman, A., Che Wel, C.A. & Mohamed, R. (2024) *The Impact of Personalization on Customers' Loyalty and the Intention to Use E-Banking Services*. *Sustainability*, 16(3), 1029. https://doi.org/10.3390/su16031029

5. Švihrová, H., Liška, M. & Haščič, M. (2025) *Designing Digital Health Interventions with Causal Inference and Multi-Armed Bandits: A Review*. *Frontiers in Digital Health*, [Preprint]. https://doi.org/10.3389/fdgth.2025.1435917

6. https://www.researchgate.net/figure/ASUM-DM-method-source-29_fig2_321944704

Appendix

1. **Data preprocessing**

```python
# Install necessary libraries
!pip install shap xgboost scikit-learn pandas matplotlib seaborn --quiet

# Import libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import shap
from sklearn.preprocessing import MinMaxScaler
from sklearn.model_selection import train_test_split
import xgboost as xgb
import warnings
warnings.filterwarnings('ignore')
```

```python
# Load data files
path = "/content/"

account = pd.read_csv(path + 'account.csv', sep=';')
client = pd.read_csv(path + 'client.csv', sep=';')
card = pd.read_csv(path + 'card.csv', sep=';')
disp = pd.read_csv(path + 'disp.csv', sep=';')
loan = pd.read_csv(path + 'loan.csv', sep=';')
order = pd.read_csv(path + 'order.csv', sep=';')
trans = pd.read_csv(path + 'trans.csv', sep=';', engine='python', on_bad_lines='skip')
district = pd.read_csv(path + 'district.csv', sep=';')
```

```python
data.shape
```

```
(309635, 18)
```

```python
data.describe()
```

	disp_id	client_id	account_id	birth_number	district_id_x	district_id_y	date_x	trans_id	date_y	amount	balance	account
count	309635.000000	309635.000000	309635.000000	309635.000000	309635.000000	309635.000000	309635.000000	3.096350e+05	309635.000000	309635.000000	309634.000000	8.332000e+04
mean	3504.516379	3532.753484	2907.605112	541317.312132	37.784902	37.825601	933519.126920	1.329771e+06	945071.722664	5922.792989	36023.783522	4.656602e+07
std	2971.231390	3043.202556	2476.123682	169290.739154	25.206674	25.556921	5763.853954	1.231737e+06	7006.121547	9489.570719	19742.925355	3.003058e+07
min	1.000000	1.000000	1.000000	120302.000000	1.000000	1.000000	930101.000000	1.000000e+00	930101.000000	0.000000	-13588.700000	0.000000e+00
25%	1395.000000	1395.000000	1159.000000	416125.000000	14.000000	13.000000	930406.000000	4.161870e+05	940606.000000	135.000000	22860.000000	1.966016e+07
50%	2932.000000	2932.000000	2427.000000	546130.000000	39.000000	39.000000	930803.000000	8.475490e+05	950116.000000	2100.000000	31024.600000	4.648518e+07
75%	4404.000000	4404.000000	3653.000000	685221.000000	61.000000	62.000000	931215.000000	2.108014e+06	950731.000000	6741.000000	44513.325000	7.232217e+07
max	13690.000000	13998.000000	11382.000000	875927.000000	77.000000	77.000000	960107.000000	3.682947e+06	960107.000000	80400.000000	193909.900000	9.998564e+07

2. XGBoost Training

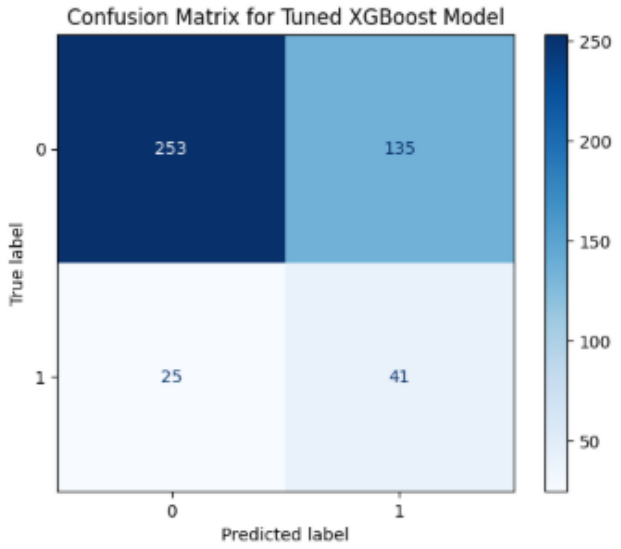
```python
from sklearn.model_selection import GridSearchCV
from xgboost import XGBClassifier
# Define the parameter grid for GridSearchCV
param_grid = {
    'n_estimators': [100, 200, 300],
    'learning_rate': [0.01, 0.05, 0.1],
    'max_depth': [3, 5, 7],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
    'gamma': [0, 0.1, 0.2],
    'objective': ['binary:logistic'],
    'eval_metric': ['logloss']
}
# Initialize an XGBClassifier
xgb_model = XGBClassifier(use_label_encoder=False, random_state=42)

# Initialize GridSearchCV
grid_search = GridSearchCV(
    estimator=xgb_model,
    param_grid=param_grid,
    scoring='recall',
    cv=3,
    verbose=1,
    n_jobs=-1  # Use all available cores
)
# Fit GridSearchCV to the SMOTE-resampled data
grid_search.fit(X_resampled, y_resampled)
# Print the best parameters and best recall score
print("Best parameters found: ", grid_search.best_params_)
print(f"Best recall score found: {grid_search.best_score_:.4f}")
# Store the best estimator
best_xgb_model = grid_search.best_estimator_
```

```
Fitting 3 folds for each of 729 candidates, totalling 2187 fits
Best parameters found:  {'colsample_bytree': 0.6, 'eval_metric': 'logloss', 'gamma': 0, 'learning_rate': 0.01, 'max_depth': 5, 'n_estimators': 100, 'objective': 'binary:logistic', 'subsample': 0.8}
```

```python
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

cm = confusion_matrix(y_test, y_pred_tuned, labels=best_xgb_model.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=best_xgb_model.classes_)
disp.plot(cmap=plt.cm.Blues)
plt.title('Confusion Matrix for Tuned XGBoost Model')
plt.show()
```



3. Contextual Bandit Module

```python
# Imports
import numpy as np
import random
from collections import defaultdict

# Define possible action types (offers) for recommendation
candidate_actions = ['Loan', 'CreditCard', 'Investment']

# Assume dimension of user context (embedding size)
d = 10  # e.g., Autoencoder output size
I_d = np.identity(d)

# Number of simulation rounds
T = 100

# Simulated user embeddings (replace with real embeddings from your model!)
user_contexts = {i: np.random.rand(d) for i in range(T)}

# Simulated ranking model (replace this with your trained XGBoost model + prediction logic)
class MockXGBModel:
    def predict(self, user, action):
        return np.random.rand()

mock_xgb_model = MockXGBModel()

```

```python
# Random user selection function
def get_next_user():
    return random.randint(0, T - 1)

# Simulated user feedback based on action type
def simulate_user_response(user, chosen_action):
    # Define different acceptance rates by action
    rates = {
        'Loan': 0.8,
        'CreditCard': 0.6,
        'Investment': 0.4
    }
    return 1 if random.random() < rates[chosen_action] else 0

# Optional: Placeholder for updating models in epsilon-greedy (not implemented here)
def update_estimated_value(chosen_action, x, reward):
    pass

```

```python
# Algorithm choice
use_linucb = True

# Parameters
epsilon = 0.1  # Epsilon for epsilon-greedy
alpha = 1.0    # Alpha for LinUCB confidence bound

# Initialize LinUCB matrices
A = {a: I_d.copy() for a in candidate_actions}
b = {a: np.zeros(d) for a in candidate_actions}

# Data tracking
action_counts = defaultdict(int)
reward_counts = defaultdict(int)

```

```python
print(f"Starting Bandit Simulation for {T} users...")

for t in range(1, T + 1):
    user = get_next_user()
    x = user_contexts[user]

    # Step 1: Get predicted scores from model (XGBoost or placeholder)
    xgb_scores = {a: mock_xgb_model.predict(user, a) for a in candidate_actions}

    # Step 2: Decide whether to explore or exploit
    if np.random.rand() < epsilon:
        # Exploration
        chosen_action = np.random.choice(candidate_actions)
    else:
        if use_linucb:
            scores = {}
            for a in candidate_actions:
                A_inv = np.linalg.inv(A[a])
                theta = A_inv.dot(b[a])
                pred_reward = theta.dot(x)
                uncertainty = alpha * np.sqrt(x.dot(A_inv).dot(x))
                scores[a] = pred_reward + uncertainty
        else:
            # Pure exploitation
            scores = {a: xgb_scores[a] for a in candidate_actions}
        chosen_action = max(scores, key=scores.get)

    # Step 3: Simulate feedback (reward)
    reward = simulate_user_response(user, chosen_action)
```